ID #: _____ Name: _____

SOTR Real-Time Linux (kind of) Cheatsheet

1. Compilation & Configuration

- **Compilation Flags:** Real-time extensions and threads require linking standard C libraries by appending `-lrt -pthread`.
- **Scheduling Classes:**
  - `SCHED_OTHER`: Default time-sharing class (Completely Fair Scheduler).
  - `SCHED_FIFO`: Fixed-priority real-time class where same-priority tasks execute in arrival order.
  - `SCHED_RR`: Fixed-priority real-time class with round-robin time-slicing.
  - `SCHED_DEADLINE`: Dynamic priority scheduling implementing Earliest Deadline First (EDF) and Constant Bandwidth Server (CBS).

2. Thread Creation & Real-Time Scheduling API

- **Thread Management:** Standard POSIX thread lifecycle is handled using `pthread_create()`, `pthread_exit()`, and `pthread_join()`.
- **Process-Level Policy:** `sched_setscheduler()` is used to set the scheduling policy and priority at the process level.
- **Task Information:** Programmatically retrieve task details using `gettid()` for the thread ID, `sched_getscheduler()` for the policy, and `sched_getparam()` for the RT priority.
- **Thread Attributes Initialization:** Prepare a thread's attributes using `pthread_attr_init()`.
- **Scheduling Inheritance:** Use `pthread_attr_setinheritsched()` with `PTHREAD_EXPLICIT_SCHED` to apply custom scheduling, or `PTHREAD_INHERIT_SCHED` to inherit the main thread's RT class and priority.
- **Setting Policy & Priority:** Define the real-time setup using `pthread_attr_setschedpolicy()` and `pthread_attr_setsc` (e.g., assigning a priority between 1 and 99).

3. Timing & Periodic Execution

- **Getting Current Time:** Read the current time into a `timespec` structure using `clock_gettime()`.
- **Periodic Sleep:** Implement precise periodic execution using `clock_nanosleep()`.

4. Memory Management

- **Memory Locking:** Prevent paging and page-fault latency by locking all current and future allocated physical memory pages into RAM using `mlockall(MCL_CURRENT | MCL_FUTURE)`.
- **Memory Unlocking:** Use `munlockall()`, `mlock()`, or `munlock()` for other memory locking operations.

ID #: _____ Name: _____

5. Synchronization & Inter-Process Communication (IPC)

- **Mutexes:** Ensure mutual exclusion using `pthread_mutex_init()`, `pthread_mutex_destroy()`, `pthread_mutex_lock()`, and `pthread_mutex_unlock()`.
- **Priority Inheritance Protocol (PiP):** Prevent priority inversion by initializing mutex attributes with `pthread_mutexatt` and applying `pthread_mutexattr_getprotocol()` with `PTHREAD_PRIO_INHERIT`.
- **Condition Variables:** Use `pthread_cond_init()`, `pthread_cond_wait()`, and `pthread_cond_broadcast()` to provide synchronized notification and avoid the use of standard signals.
- **POSIX Message Queues:** Used to emulate distributed hardware systems by transferring state data and control commands between tasks. Ideal for TT+ET architectures to trigger sporadic tasks upon data arrival.
  Standard API: `mq_open()`, `mq_send()`, `mq_receive()`, `mq_close()`, `mq_unlink()`.
- **POSIX Semaphores:** Used to orchestrate strict execution sequences and synchronize tasks, often paired with Shared Memory for lower-overhead data transfer. Highly preferred over standard Linux signals in RT applications.
  Standard API: `sem_init()`, `sem_wait()`, `sem_post()`, `sem_destroy()`.
- **Shared Memory:** Provides the lowest-overhead IPC method for passing data between tasks in a control loop, but requires explicit protection (via Mutexes or Semaphores) to prevent data corruption.

6. CPU Affinity

- **Defining Sets:** Isolate tasks to specific CPUs to reduce interference by clearing a mask with `CPU_ZERO()` and adding target cores with `CPU_SET()`.
- **Applying Affinity:** Bind a process or thread to the defined CPU set using the `sched_setaffinity()` syscall.
- **Hardware Interrupt Routing:** Interrupt CPU affinity can be managed via files such as `/proc/irq/default_smp_affini` or `/proc/irq/irq-number/smp_affinity`.

7. CLI & Performance Evaluation Tools

- **Priority Modification CLI:** `chrt` can be used to query or set RT policies and priorities directly from the terminal (e.g., `chrt -f 99 ./myprog`).
- **Process Monitoring:** Use `top` or `htop` (`top -H` for threads) to view running tasks; note that RT priorities display as negative values offset by 1 (e.g., RT priority 25 shows as -26).
- **Performance Counters:** `perf` allows counting hardware and software events like cache misses, page faults, and context switches.
- **Kernel Tracing:** `ftrace` tracks kernel function execution. Tools like `trace-cmd` simplify ftrace usage, while `kernelshark` provides a GUI for visualizing the trace data.
- **Synthetic Workload:** `stress-ng` is used to inject controlled stress/workloads to verify real-time task schedulability and latencies under heavy load.